Quant101

A Quantitative Trader's Learning Lab

From Beginner to Job-Market-Ready Quant Researcher

Jerry Hong
with AI-assisted authoring

Started September 2025
Last updated: March 4, 2026

Preface

This document is a **learning lab notebook** — not a textbook, not API documentation. It records my path from a beginning trader to a quant researcher, maintained together with an AI agent who serves as a pair-programming partner and knowledge editor.

The companion codebase, `quant101`, started in September 2025 as a backtesting experiment built on Polygon.io[1] flat-file data. It has since grown into a data pipeline, backtest engine, strategy framework, and live-monitor system.

What This Document Is

- A **guided build sequence**: each chapter asks you to build something, then measure it, then learn the theory that explains the measurement.
- A **depth-first** exploration: fewer topics, deeper treatment, real experiments on real data.
- A **job-market bridge**: the skill progression follows what quant interviews actually test — not academic curriculum order.

What This Document Is Not

- Not a reference manual for current file paths or API signatures — those change with every refactor.
- Not a textbook that front-loads 200 pages of theory before any code.
- Not a survey of every quant topic — Wikipedia exists.

How to Read

Every chapter follows the same **lab module** structure:

1. **Motivation** — Why this matters for the project and for interviews.
2. **Concepts** — Just enough theory, introduced when you need it.
3. **Implementation** — What to build, described at the interface level (not hardcoded paths — the architecture evolves).
4. **Experiment** — Run it on real data. Record what you find.
5. **Reflection** — What worked, what broke, what to read next.

Chapters are ordered by **build dependency**: you can read straight through and each chapter assumes only the previous ones. Mathematical foundations live in Appendix A and are cross-referenced when needed — flip there when a chapter says "see Appendix A."

[1] Rebranded as Massive around October 2025: https://massive.com/blog/polygon-is-now-massive

Skill Progression

Level	Parts	You Can...
Beginner	I, II	Build a data pipeline, run a backtest, measure performance
Intermediate	III	Construct & evaluate alpha factors, think cross-sectionally
Advanced	IV, V	Build portfolios, model risk, apply ML with proper validation

Part III is the **inflection point**: where a "trader who codes" becomes a "quant researcher." The document spends the most depth there.

Contents

Preface **1**

I Data Engineering **6**

1 Data Pipeline Design **8**
- 1.1 Motivation . . . 8
- 1.2 Concepts . . . 8
  - 1.2.1 The Four-Stage Pipeline . . . 8
  - 1.2.2 Columnar Storage . . . 8
  - 1.2.3 Incremental vs. Full Refresh . . . 9
- 1.3 Implementation Sketch . . . 9
- 1.4 Experiment: Pipeline Profiling . . . 9

2 Data Quality and Corporate Actions **10**
- 2.1 Motivation . . . 10
- 2.2 Concepts . . . 10
  - 2.2.1 Stock Splits and Adjustment . . . 10
  - 2.2.2 Ticker Identity Over Time . . . 10
  - 2.2.3 The Low-Volume Ticker Problem . . . 11
- 2.3 Implementation Sketch . . . 11
- 2.4 Experiment: Split Adjustment Validation . . . 11

3 Universe Construction **12**
- 3.1 Motivation . . . 12
- 3.2 Concepts . . . 12
  - 3.2.1 Survivorship Bias . . . 12
  - 3.2.2 Liquidity Filters . . . 12
  - 3.2.3 Sector and Industry Mapping . . . 13
- 3.3 Implementation Sketch . . . 13
- 3.4 Experiment: Survivorship Bias Quantification . . . 13

II Strategy and Backtesting 14

4 Technical Indicators and Signal Design 16
4.1 Motivation . . . 16
4.2 Concepts . . . 16
4.2.1 Indicator Taxonomy . . . 16
4.2.2 The Registry Pattern . . . 16
4.3 Implementation Sketch . . . 17

5 Rule-Based Strategies 18
5.1 Motivation . . . 18
5.2 Concepts . . . 18
5.2.1 Strategy as a State Machine . . . 18
5.2.2 The Strategy Base Class . . . 18
5.2.3 Case Study: BBIBOLL . . . 19

6 Backtest Engine and Performance Metrics 20
6.1 Motivation . . . 20
6.2 Concepts . . . 20
6.2.1 The Backtest Loop . . . 20
6.2.2 Return Metrics . . . 20
6.2.3 Trade-Level Metrics . . . 21
6.3 Implementation Sketch . . . 21

7 Backtest Bias and Validation 22
7.1 Motivation . . . 22
7.2 Concepts . . . 22
7.2.1 The Three Cardinal Biases . . . 22
7.2.2 Walk-Forward Validation . . . 22
7.2.3 Statistical Significance of Backtest Results . . . 23
7.3 Experiment: How Fragile Is the BBIBOLL Sharpe? . . . 23

III Alpha Research 24

8 From Rules to Factors 26
8.1 Motivation . . . 26
8.2 Concepts . . . 26
8.2.1 Time-Series vs. Cross-Sectional Thinking . . . 26
8.2.2 The Alpha Pipeline . . . 27

9 Factor Evaluation 28
9.1 Motivation . . . 28
9.2 Concepts . . . 28
9.2.1 Information Coefficient (IC) . . . 28
9.2.2 Information Ratio (IR) . . . 28
9.2.3 IC Decay . . . 29
9.2.4 Turnover . . . 29
9.3 Implementation Sketch . . . 29
9.4 Experiment: BBIBOLL Factor Evaluation . . . 30

10 Factor Construction 31
10.1 Motivation . . . 31

10.2 Concepts . . . 31
10.2.1 Normalization . . . 31
10.2.2 Neutralization . . . 31
10.2.3 Orthogonalization . . . 32
10.3 Implementation Sketch . . . 32
10.4 Experiment: Neutralization Impact . . . 32

11 Factor Combination **33**
11.1 Motivation . . . 33
11.2 Concepts . . . 33
11.2.1 Equal-Weight and IC-Weight . . . 33
11.2.2 Mean-Variance on Factor ICs . . . 33
11.2.3 Risk Parity . . . 33
11.3 Experiment: Two-Factor Composite . . . 34

IV Portfolio, Risk, and Execution **35**

12 Risk Measures **37**
12.1 Motivation . . . 37
12.2 Concepts . . . 37
12.2.1 Value at Risk . . . 37
12.2.2 Distribution Properties of Returns . . . 37

13 Portfolio Construction **39**
13.1 Motivation . . . 39
13.2 Concepts . . . 39
13.2.1 Factor Exposure and CAPM . . . 39
13.2.2 Position Sizing . . . 39

14 Execution and Transaction Costs **40**
14.1 Motivation . . . 40
14.2 Concepts . . . 40
14.2.1 Cost Components . . . 40
14.2.2 Execution Algorithms . . . 40
14.2.3 Cost Modeling in Backtests . . . 40
14.3 Experiment: Cost Sensitivity Analysis . . . 41

V Machine Learning in Quantitative Finance **42**

15 Feature Engineering for Finance **44**
15.1 Motivation . . . 44
15.2 Concepts . . . 44
15.2.1 Feature Categories . . . 44
15.2.2 Feature Processing . . . 44

16 Time-Series Validation **46**
16.1 Motivation . . . 46
16.2 Concepts . . . 46
16.2.1 Validation Schemes . . . 46

17 Tree Models and Ensembles **47**

17.1 Motivation . . . 47
17.2 Concepts . . . 47
17.2.1 Why Trees for Finance? . . . 47
17.2.2 Ensemble Methods . . . 47
17.2.3 Hyperparameter Discipline . . . 47
17.3 Experiment: LightGBM vs. Linear Factor Model . . . 48

A Mathematical Reference **49**
A.1 Probability and Statistics . . . 49
A.1.1 Key Theorems . . . 49
A.1.2 Hypothesis Testing . . . 50
A.1.3 Regression . . . 50
A.2 Time Series . . . 50
A.3 Stochastic Calculus . . . 51

B Quant Notes **52**

C Glossary **60**

D Learning Priority and Reading List **61**
D.1 Recommended Order . . . 61
D.2 Suggested Reading . . . 61

Part I

Data Engineering

All research starts from clean data. No analysis is trustworthy if the data pipeline is broken. This part covers the methodology of acquiring, cleaning, and organizing financial market data — the foundation everything else rests on.

Chapter 1

Data Pipeline Design

1.1 Motivation

Most quant tutorials skip straight to "load a CSV and compute moving averages." In practice, 60–80% of a quant researcher's time is data work: downloading, parsing, validating, adjusting, and caching. If you can't trust your data, every signal you build and every backtest you run is unreliable.

This chapter covers the *design* of a market data pipeline — the stages, the tradeoffs, and the failure modes — without prescribing a fixed directory layout (that evolves with the codebase).

Interview relevance: "Walk me through how you'd build a data pipeline for a factor research platform" is a common quant infrastructure question.

1.2 Concepts

1.2.1 The Four-Stage Pipeline

Every market data system, whether at a hedge fund or on a personal laptop, follows the same logical stages:

1. **Acquisition** — Download raw data from external sources.
2. **Normalization** — Parse, schema-map, and convert to an efficient columnar format.
3. **Enrichment** — Apply corporate actions (splits, dividends), map identifiers, filter the universe.
4. **Serving** — Load clean, adjusted data into research or backtest code with minimal latency.

Remark 1.1. The four stages are a logical decomposition. In a small project they may live in two scripts; at a fund they may be four separate services with message queues between them. The *boundaries* matter more than the implementation.

1.2.2 Columnar Storage

Definition 1.1 (Columnar Format)**.** A storage layout where values for each column are stored contiguously on disk. Examples: Apache Parquet, Apache Arrow, ORC.

For OHLCV data with N rows and 6 columns, a row-oriented CSV reads all 6 columns even when you only need `close`. Parquet reads only the `close` column — typically 5–10× faster for analytical queries.

Remark 1.2. This project uses Parquet as the canonical on-disk format and Polars (which reads Parquet natively via Arrow) as the in-memory engine. The choice is deliberate: Polars' lazy evaluation and predicate pushdown make it far more efficient than Pandas for the scan-filter-aggregate pattern that dominates quant data work.

1.2.3 Incremental vs. Full Refresh

Two update strategies:

- **Full refresh**: Re-download and re-process everything. Simple but wasteful. Acceptable for small datasets or monthly rebuilds.
- **Incremental update**: Only fetch new data since the last run. Requires tracking watermarks (last processed date). More complex but essential when daily data grows to millions of rows.

The project uses incremental updates for daily aggregates (fetch only recent N days) and full refresh for slowly-changing reference data (ticker lists, split events).

1.3 Implementation Sketch

The pipeline is implemented as a sequence of idempotent scripts, orchestrated by a shell-based task runner. The key design decisions:

- **Configuration-driven**: A single YAML config file (`basic_config.yaml`) defines the data root directory, update mode (standalone / server / client), and sync settings. The config is gitignored; a committed `.example` template documents the schema.
- **Idempotent scripts**: Each script can be re-run safely. Re-downloading an already-downloaded date is a no-op; re-converting an already-converted file overwrites with identical output.
- **Mode-aware orchestration**: A single entry-point script detects the update mode and runs the appropriate task subset (e.g., a client skips API fetches and syncs metadata from a server instead).

Lab Module 1.1. Build it: Set up the pipeline end-to-end for one asset class (e.g., US equities daily aggregates). Download → convert → verify row counts → load into a Polars DataFrame and compute a 20-day SMA. Time each stage. Where is the bottleneck?

1.4 Experiment: Pipeline Profiling

Experiment 1.1. **Goal**: Profile each stage of the pipeline for 7 days of US equity daily aggregates (∼10,000 tickers × 7 days).

Method: Time each stage separately — download, CSV.gz → Parquet conversion, and Polars lazy scan + collect.

Record: Wall-clock time per stage, file sizes (CSV.gz vs. Parquet), memory usage during conversion.

Reflection space: *(fill in after running)*

Chapter 2

Data Quality and Corporate Actions

2.1 Motivation

Raw market data is *never* clean. Stock splits change historical prices, tickers get reused by different companies, and data vendors introduce errors. Ignoring these issues doesn't make them go away — it makes your backtest return a fantasy number.

This is the unglamorous foundation that separates a toy project from a usable research platform.

Interview relevance: "How do you handle stock splits in your data?" and "What is survivorship bias at the data level?" are standard screening questions.

2.2 Concepts

2.2.1 Stock Splits and Adjustment

Definition 2.1 (Split Adjustment)**.** A stock split with ratio r (e.g., $r = 2$ for a 2-for-1 split) requires adjusting all historical prices before the split date:

$$P_{\text{adj}} = P_{\text{raw}} \cdot \frac{1}{r}$$

Volume is adjusted inversely: $V_{\text{adj}} = V_{\text{raw}} \cdot r$.

Without adjustment, a 2-for-1 split looks like a 50% crash in the price series, which poisons every downstream calculation.

Common Pitfall 2.1. **Split data has errors.** Vendor-provided split tables contain incorrect ratios, missing events, and duplicate entries. The project addresses this with a CSV-based error correction layer: a manually curated file lists split IDs to remove (erroneous) and add (missing), applied as a filter before adjustment. This pattern — *override layer on top of vendor data* — is standard practice at production quant firms.

2.2.2 Ticker Identity Over Time

Definition 2.2 (FIGI (Financial Instrument Global Identifier))**.** A 12-character, alphanumeric identifier assigned to financial instruments. Unlike ticker symbols, FIGIs are globally unique and persistent across name changes, mergers, and exchange transfers.

The ticker identity problem: company A trades as `AAPL`, gets acquired, and later company B is listed under the same symbol `AAPL`. Using ticker strings alone conflates two unrelated entities.

The solution is a **connected-component algorithm** on FIGI fields (`composite_figi` and `share_class_figi`). Two ticker records sharing any FIGI are the same instrument. The algorithm runs on the full ticker universe and produces a canonical mapping from every historical ticker string to a unique entity ID.

2.2.3 The Low-Volume Ticker Problem

Experiment 2.1. **Observation**: Many tickers show extended zero-volume periods (> 50 consecutive trading days). Investigation revealed these are mostly relisted instruments or ticker-symbol reuse by shell companies.

Decision: Skip these tickers in backtest universes. This is a documented data quality tradeoff — not a silent exclusion.

Lesson: Every universe filter introduces a bias. Documenting *why* you exclude something is as important as excluding it.

2.3 Implementation Sketch

- **Split adjustment**: Applied during the data-serving stage, not during conversion. This preserves raw data fidelity and allows re-adjustment when corrections are added.
- **Error correction**: A pair of CSV files per data type (e.g., `splits_error.csv`) with columns `[id, error_type, ...]`. The loader reads vendor data, filters by the error file, then serves the corrected result.
- **Ticker mapping**: The FIGI connected-component algorithm runs once per universe refresh and caches the mapping. Downstream code references entity IDs, not raw ticker strings.

Lab Module 2.1. Build it: Take a known split event (e.g., NVDA 10-for-1, June 2024). Load raw and adjusted price series. Plot both on the same chart. Verify the adjustment formula by hand for one data point.

2.4 Experiment: Split Adjustment Validation

Experiment 2.2. **Goal**: Validate the split adjustment pipeline against a known event.

Method: For NVDA's 10-for-1 split (2024-06-10):

1. Load unadjusted close prices around the split date.
2. Apply the split adjustment.
3. Compare adjusted close with the value from a trusted source (e.g., Yahoo Finance adjusted close).
4. Compute the percentage difference — it should be $< 0.01\%$.

Reflection space: *(fill in after running)*

Chapter 3

Universe Construction

3.1 Motivation

"What stocks do you include in your backtest?" sounds simple. It isn't. The choice of universe determines whether your results generalize or are an artifact of selection.

A universe that is too narrow (only FAANG stocks) overfits to a handful of names. A universe that is too broad (all 15,000 US tickers including OTC penny stocks) drowns signals in noise and introduces illiquidity artifacts.

Interview relevance: "How did you construct your backtest universe?" is a question that separates candidates who understand survivorship bias from those who don't.

3.2 Concepts

3.2.1 Survivorship Bias

Definition 3.1 (Survivorship Bias)**.** The error of only including assets that *survived* to the present date in a historical analysis. Delisted stocks (bankruptcies, acquisitions, regulatory removals) are excluded, biasing aggregate returns upward.

A backtest universe must be constructed **as-of** each historical date, including stocks that existed at that date even if they were later delisted.

3.2.2 Liquidity Filters

Common filters for a "tradeable" universe:

- **Minimum price**: Exclude stocks below \$1–\$5 (penny stock noise).
- **Minimum volume**: Exclude stocks with average daily volume below a threshold (e.g., 100,000 shares) — you can't trade what you can't fill.
- **Market cap**: Exclude micro-caps if your strategy can't handle the illiquidity and volatility.
- **Listing exchange**: Exclude OTC/pink-sheet stocks for most strategies.

Remark 3.1. Every filter is a tradeoff. Excluding small-caps removes noise but also removes potential alpha. The filter parameters should be *explicit and documented*, not buried in a data loader.

3.2.3 Sector and Industry Mapping

Cross-sectional analysis (Part III) requires grouping stocks by sector. Standard classifications:

- **GICS** (Global Industry Classification Standard): 11 sectors, 24 industry groups — the institutional standard.
- **SIC / NAICS**: Older classification systems, still used in SEC filings.

Sector membership is time-varying (companies change sectors after M&A or business pivots). A proper implementation uses point-in-time sector assignments.

3.3 Implementation Sketch

A universe construction module should provide:

```python
def build_universe(
    as_of_date: date,
    min_price: float = 1.0,
    min_avg_volume: int = 100_000,
    exclude_otc: bool = True,
) -> list[str]:
    """
    Return a list of tradeable tickers as-of a given date.
    Point-in-time: includes stocks that were active on as_of_date,
    even if they were later delisted.
    """
```

The key constraint is **point-in-time correctness**: the function must not use information from after `as_of_date`.

Lab Module 3.1. Build it: Implement a minimal universe filter. Compare the number of tickers in the filtered universe vs. the full ticker list at several historical dates. How much does the universe size change over time?

3.4 Experiment: Survivorship Bias Quantification

Experiment 3.1. **Goal**: Measure the impact of survivorship bias on a simple momentum strategy.
Method:

1. Run a 12-month momentum strategy on today's S&P 500 constituents (survivorship-biased universe).
2. Run the same strategy on the point-in-time universe (historical constituents at each rebalance date).
3. Compare CAGR and Sharpe ratio.

Expected: The biased backtest will show higher returns (failed companies excluded). The difference is the "survivorship premium."
Reflection space: *(fill in after running)*

Part II

Strategy and Backtesting

A backtest is a simulation of what would have happened *if you had traded a strategy in the past. It is the primary tool for evaluating ideas — and the primary source of self-deception if done carelessly. This part covers how to design strategies, measure performance, and avoid the biases that make bad strategies look good.*

Chapter 4

Technical Indicators and Signal Design

4.1 Motivation

An indicator transforms raw price/volume data into a derived quantity that is (hopefully) easier to make trading decisions from. Every rule-based strategy starts with indicators; every factor-based strategy (Part III) is built from them.

The engineering question is not "which indicator is best" — it is "how do I organize indicators so they are composable, testable, and reusable across strategies?"

Interview relevance: You won't be asked to recite indicator formulas. You *will* be asked: "How is your indicator pipeline structured? Can you add a new indicator without touching the strategy code?"

4.2 Concepts

4.2.1 Indicator Taxonomy

- **Trend**: Moving averages (SMA, EMA, BBI), MACD, ADX — capture the direction and strength of price movement.
- **Mean reversion**: Bollinger Bands, RSI, z-score of price relative to a moving average — capture deviation from "normal."
- **Volume**: OBV, VWAP, Accumulation/Distribution — capture the conviction behind price moves.
- **Volatility**: ATR, Bollinger bandwidth, realized volatility — capture the magnitude of price fluctuations.

4.2.2 The Registry Pattern

A clean indicator system uses a **registry**: each indicator registers itself via a decorator, and the strategy code requests indicators by name without knowing their implementation details.

```python
@register_indicator("bbiboll")
def compute_bbiboll(df: pl.DataFrame) -> pl.DataFrame:
    """Compute BBI + Bollinger deviation percentile rank."""
    ...
```

Benefits:

- Adding a new indicator requires zero changes to existing code.
- Indicators can declare whether they need per-ticker grouping.
- The registry enables bulk computation: "apply all registered indicators to this DataFrame."

4.3 Implementation Sketch

The indicator subsystem has three layers:

1. **Registry** — a global dictionary mapping names to functions, populated by decorator registration.
2. **Base applicator** — a helper that applies a registered indicator function over grouped data (group-by-ticker).
3. **Indicator modules** — one file per indicator (or family), each file auto-imported to trigger registration.

Lab Module 4.1. Build it: Implement a new indicator (e.g., RSI or ATR) using the registry pattern. Verify it works by applying it to a single ticker's OHLCV DataFrame, then to the full universe via the group-by applicator. Confirm that no strategy code needed changes.

Chapter 5

Rule-Based Strategies

5.1 Motivation

Before building factor models or ML pipelines, every quant should build at least one rule-based strategy end to end: from signal definition through execution logic to PnL calculation. This exercises the full stack and reveals problems (data bugs, look-ahead leaks, position tracking errors) that are invisible in theory.

5.2 Concepts

5.2.1 Strategy as a State Machine

A rule-based strategy is a **state machine** with states {Flat, Long, Short} and transitions triggered by indicator conditions:

- **Entry signal**: Condition that triggers opening a position (e.g., BBI deviation crosses below the 10th percentile ⇒ buy).
- **Exit signal**: Condition that triggers closing (e.g., deviation returns above the 50th percentile, or stop-loss hit).
- **Position sizing**: How much capital to allocate per trade (equal weight, volatility-targeted, Kelly — see Chapter 13).

5.2.2 The Strategy Base Class

An abstract base class enforces a contract:

```python
class StrategyBase(ABC):
    @abstractmethod
    def generate_signals(self, data: pl.DataFrame) -> pl.DataFrame:
        """Return a DataFrame with a 'signal' column."""

    def run(self, data: pl.DataFrame) -> pl.DataFrame:
        """Apply signals, compute positions, return trades."""
```

Every concrete strategy inherits this interface. The backtest engine only depends on the base class — it doesn't know whether the strategy uses BBIBOLL, SMA crossover, or a neural network.

5.2.3 Case Study: BBIBOLL

The project's first complete strategy combines two ideas:

1. **BBI (Bull and Bear Index)**: The average of four SMAs (3, 6, 12, 24 periods). Smoother than any single MA.
2. **Bollinger deviation**: The distance of price from BBI, expressed as a percentile rank within a rolling window. Low percentile = price is unusually far below the mean = potential mean-reversion buy signal.

Entry: deviation percentile < threshold. Exit: percentile recovers, or stop-loss / take-profit triggers.

Remark 5.1. BBIBOLL is a **time-series signal**: computed from each stock's own history. In Part III, we will reframe it as a **cross-sectional factor** by ranking the signal across all stocks at each date. This reframing is the key conceptual leap from "strategy" to "alpha."

Lab Module 5.1. Build it: Implement a minimal SMA-crossover strategy using the strategy base class interface. Run it on one ticker for one year. Inspect the trades DataFrame: are entry/exit dates correct? Does position tracking handle edge cases (e.g., signal on the last day)?

Chapter 6

Backtest Engine and Performance Metrics

6.1 Motivation

The engine is the machinery that turns a strategy's signals into simulated equity curves. The performance metrics are the language you use to describe whether the results are good, bad, or meaningless.

Both are mechanical — the intellectual challenge is in the *next* chapter (bias and validation). But you need the mechanics first.

6.2 Concepts

6.2.1 The Backtest Loop

At its core, a backtest iterates over time steps and applies:

1. **Signal generation**: Run the strategy on data available up to time t.
2. **Order creation**: Translate signals into hypothetical orders.
3. **Execution simulation**: Fill orders at assumed prices (e.g., next-day open) with assumed costs.
4. **Portfolio update**: Update positions, cash, and equity.

Common Pitfall 6.1. **Look-ahead in the loop.** The most common backtest bug: using the close price at time t to generate a signal and then assuming you can execute at that same close price. In reality, you can only act on information *after* observing it — execution should happen at $t + 1$ open at the earliest.

6.2.2 Return Metrics

Definition 6.1 (Sharpe Ratio)**.**

$$SR = \frac{\mathbb{E}[r_p - r_f]}{\sigma(r_p - r_f)}$$

Annualized: $SR_{\text{annual}} = SR_{\text{daily}} \times \sqrt{252}$.

The Sharpe ratio is the single most quoted performance metric. It measures **risk-adjusted return**: how much return you earn per unit of volatility. See Appendix A.1 for the underlying statistics.

Definition 6.2 (Sortino Ratio)**.** Like Sharpe, but only penalizes **downside** volatility:

$$\text{Sortino} = \frac{\mathbb{E}[r_p - r_f]}{\sigma_{\text{downside}}}$$

Preferred when returns are asymmetric (positive skew $\Rightarrow$ Sortino $>$ Sharpe).

Definition 6.3 (Maximum Drawdown)**.**

$$MDD = \max_t \left(\frac{\text{Peak}_t - \text{Trough}_t}{\text{Peak}_t} \right)$$

The largest peak-to-trough decline. For a live trader, MDD is the metric that determines whether you get fired or your fund gets redeemed.

Definition 6.4 (CAGR and Calmar Ratio)**.**

$$CAGR = \left(\frac{V_T}{V_0} \right)^{1/T} - 1 \qquad \text{Calmar} = \frac{CAGR}{|MDD|}$$

6.2.3 Trade-Level Metrics

Definition 6.5 (Win Rate and Payoff Ratio)**.**

$$\text{Win Rate} = \frac{\text{winning trades}}{\text{total trades}} \tag{6.1}$$

$$\text{Payoff Ratio} = \frac{\text{average win}}{\text{average loss}} \tag{6.2}$$

A strategy can be profitable with win rate $< 50\%$ if the payoff ratio is high (trend following), or with low payoff ratio if win rate is high (mean reversion).

6.3 Implementation Sketch

The engine consists of three decoupled components:

1. **Runner**: Wires strategy + data + analyzer; produces the equity curve.
2. **PerformanceAnalyzer**: Computes all metrics from the equity curve and trade log.
3. **Visualizer**: Plots equity curves, drawdowns, monthly heatmaps, and candlestick charts with signal overlays.

Lab Module 6.1. Build it: Run the BBIBOLL strategy on a rolling set of weekly start dates (60+ windows). Collect the Sharpe ratio from each window. Plot the distribution of Sharpe ratios. Is the strategy consistently positive, or does a few good windows dominate?

Chapter 7

Backtest Bias and Validation

7.1 Motivation

This is the most important chapter in Part II. A backtest that looks good but contains biases is worse than no backtest at all — it gives false confidence that costs real money.

Every quant interview will probe your understanding of backtest biases. Saying "my backtest Sharpe is 2.5" without being able to explain *why it might be wrong* is a red flag.

7.2 Concepts

7.2.1 The Three Cardinal Biases

Definition 7.1 (Look-Ahead Bias)**.** Using information at time t that was not actually available at time t.

Examples:

- Using adjusted prices that incorporate *future* splits.
- Using the full-sample mean to normalize signals.
- Generating a signal at close and executing at the same close.

Definition 7.2 (Survivorship Bias)**.** Only backtesting on assets that survived to the present day. Delisted stocks (often the worst performers) are excluded, biasing returns upward.

See Chapter 3 for how universe construction addresses this.

Definition 7.3 (Data Snooping / Overfitting)**.** Testing many parameter combinations on the same dataset and selecting the best result. The more combinations you test, the more likely one will look good *by chance*.

Deflated Sharpe Ratio (Bailey & López de Prado, 2014): adjusts the Sharpe ratio for the number of strategies tested.

7.2.2 Walk-Forward Validation

Definition 7.4 (Walk-Forward Optimization (WFO))**.**

1. Split data into rolling windows: [train | test].
2. Optimize parameters on the train window.
3. Evaluate on the test window (**no re-optimization**).
4. Slide the window forward and repeat.

5. Concatenate all test-window results for the "true" out-of-sample performance.

Remark 7.1. The project's rolling weekly backtest (60+ date windows with fixed parameters) resembles walk-forward testing but is *not* true WFO because parameters are not re-optimized inside each training window. Upgrading to true WFO is a planned enhancement.

7.2.3 Statistical Significance of Backtest Results

Definition 7.5 (t-Statistic of the Sharpe Ratio)**.** Is the backtest Sharpe distinguishable from zero?

$$t = SR \times \sqrt{n}$$

where n is the number of return periods. A rule of thumb: $|t| > 2$ (equivalently, $p < 0.05$) suggests the result is unlikely due to chance alone.

See Appendix A.1 for the underlying hypothesis testing framework.

Common Pitfall 7.1. A statistically significant Sharpe does not mean an **economically** significant strategy. A Sharpe of 0.3 with $|t| = 2.5$ (from 20 years of data) is "real" but may not cover transaction costs.

7.3 Experiment: How Fragile Is the BBIBOLL Sharpe?

Experiment 7.1. **Goal**: Stress-test the BBIBOLL strategy's reported Sharpe ratio.

Method:

1. Compute the Sharpe from the full sample.
2. Compute the t-statistic. Is $|t| > 2$?
3. Remove the best 5 trading days. How much does the Sharpe drop?
4. Run on a different time period (out-of-sample). Does the Sharpe survive?
5. Shuffle trade entry dates randomly (permutation test). What Sharpe do you get from pure randomness?

Reflection space: *(fill in after running)*

Part III

Alpha Research

This is the inflection point. Parts I and II make you a "trader who codes." Part III makes you a "quant researcher." The core shift: from asking "does this strategy make money?" to asking "does this signal predict returns, and how do I measure that rigorously?"

Chapter 8

From Rules to Factors

8.1 Motivation

In Part II you built a rule-based strategy: "if deviation percentile < 10, buy." This works, but it has a problem: the threshold is arbitrary, the evaluation is binary (profitable or not), and there is no principled way to combine it with other ideas.

The factor framework solves this. Instead of asking "should I buy?", you ask: "across all 5,000 stocks today, does a higher deviation percentile predict higher future returns?"

This reframing unlocks:

- **Continuous evaluation**: IC measures the strength of the signal, not just "profitable or not."
- **Composability**: Multiple factors can be combined into a single portfolio signal.
- **Decomposition**: You can attribute portfolio returns to individual factors.

Interview relevance: This chapter covers the exact vocabulary of quant researcher interviews. "What is the IC of your factor?" is the first question.

8.2 Concepts

8.2.1 Time-Series vs. Cross-Sectional Thinking

Definition 8.1 (Time-Series Signal)**.** A signal computed from a single asset's own history. Example: 20-day momentum return for AAPL, BBIBOLL deviation for NVDA. The signal value for AAPL does not depend on any other stock.

Definition 8.2 (Cross-Sectional Signal (Factor))**.** A signal that **ranks or normalizes** values across the entire stock universe at each point in time. Example: the z-score of PE ratio across all stocks on a given date.

The key property: the factor value for stock i on date t depends on *all other stocks* on that date.

The BBIBOLL deviation percentile is a time-series signal. To use it cross-sectionally:

$$\text{factor}_{i,t} = \frac{\text{deviation}_{i,t} - \overline{\text{deviation}_t}}{\sigma(\text{deviation}_t)}$$

where the mean and standard deviation are computed across all stocks i at date t. This produces a z-scored cross-sectional factor.

8.2.2 The Alpha Pipeline

The standard factor research workflow:

1. **Signal construction**: Compute a raw signal for each stock at each date.
2. **Normalization**: Z-score or rank-transform cross-sectionally.
3. **Evaluation**: Compute IC, IR, decay, turnover.
4. **Refinement**: Neutralize for sector, size, or other known factors.
5. **Combination**: Blend with other factors into a composite signal.
6. **Portfolio construction**: Map the composite signal to positions (Part IV).

Each step is a separate module. This chapter covers steps 1–2; the next three chapters cover steps 3–5.

Lab Module 8.1. Build it: Take the BBIBOLL deviation signal (a time-series signal) and convert it to a cross-sectional factor. At each date, z-score the deviation across all stocks in the liquid universe (Chapter 3). Plot the distribution of factor values on a single date — is it approximately normal?

Chapter 9

Factor Evaluation

9.1 Motivation

You've built a factor. Is it any good? "Running a backtest" is not the answer — backtests conflate signal quality with portfolio construction, execution, and luck. Factor evaluation isolates the question: **does this signal predict future returns?**

This chapter covers the four metrics that every quant uses to answer that question.

9.2 Concepts

9.2.1 Information Coefficient (IC)

Definition 9.1 (IC)**.** The cross-sectional rank correlation (Spearman's ρ) between a factor's signal values and subsequent forward returns:

$$IC_t = \rho_{\text{rank}}(\text{signal}_t,\ r_{t \to t+h})$$

where the correlation is computed across all stocks in the universe at date t, and h is the forecast horizon (e.g., 5 trading days).

Remark 9.1. Why Spearman (rank) and not Pearson?

- Robust to outliers (extreme returns, extreme signal values).
- Does not assume a linear relationship.
- Invariant to monotonic transformations of the signal.

In practice, you should compute *both* and compare. A large gap between Pearson and Spearman IC suggests outlier sensitivity.

9.2.2 Information Ratio (IR)

Definition 9.2 (IR)**.**

$$IR = \frac{\overline{IC}}{\sigma(IC)}$$

where $\overline{IC}$ is the time-series mean of IC_t and $\sigma(IC)$ is its standard deviation.

IR measures the **consistency** of predictive power. A factor with $|\overline{IC}| = 0.03$ and $IR = 0.5$ is far more valuable than one with $|\overline{IC}| = 0.05$ and $IR = 0.2$. Consistency beats magnitude.

Remark 9.2. Rule of thumb (Qian, Hua & Sorensen):

$\lvert IR \rvert$	Interpretation
> 0.5	Strong, tradeable factor
0.3–0.5	Decent, worth including in a composite
< 0.3	Weak or unreliable

9.2.3 IC Decay

Definition 9.3 (IC Decay Curve)**.** Plot $\overline{IC}(h)$ for increasing forecast horizons $h = 1, 2, 5, 10, 20$ days. The shape reveals the signal's natural trading frequency:

- **Fast decay** (IC drops sharply after 1–2 days): short-term signal, requires frequent trading.
- **Slow decay** (IC persists for weeks): medium-frequency signal, lower turnover.
- **Non-monotonic decay**: possible look-ahead bias or seasonal effects — investigate.

Definition 9.4 (Half-Life)**.** The number of periods until the IC drops to half its peak value. Approximate from the AR(1) coefficient ϕ of the IC time series:

$$t_{1/2} = -\frac{\ln 2}{\ln |\phi|}$$

9.2.4 Turnover

Definition 9.5 (Factor Turnover)**.** How much the factor's ranking changes from one period to the next:

$$\text{Turnover}_t = \frac{1}{N} \sum_{i=1}^{N} |w_{i,t} - w_{i,t-1}|$$

where $w_{i,t}$ is the normalized factor weight for stock i at time t.

High turnover $\Rightarrow$ high transaction costs $\Rightarrow$ the factor needs a higher gross IC to be profitable after costs.

Definition 9.6 (Capacity)**.** The maximum AUM a factor can support before market impact erodes its alpha. Small-cap momentum: high IC, low capacity. Large-cap value: lower IC, high capacity. Capacity is a function of the liquidity of the stocks the factor trades.

9.3 Implementation Sketch

A factor evaluation module should provide:

```python
class FactorAnalyzer:
    def __init__(self, signal: pl.DataFrame, returns: pl.DataFrame):
        """
        signal: (date, ticker, signal_value)
        returns: (date, ticker, forward_return_1d, ..., forward_return_20d)
        """

    def ic_series(self, horizon: int = 5) -> pl.DataFrame:
        """Cross-sectional Spearman IC at each date."""

    def ir(self, horizon: int = 5) -> float:
        """Mean IC / Std IC."""

```

```python
    def ic_decay(self, horizons: list[int]) -> pl.DataFrame:
        """Mean IC at each horizon."""

    def turnover(self) -> pl.DataFrame:
        """Daily factor turnover."""
```

Lab Module 9.1. Build it: Implement `FactorAnalyzer` with at least `ic_series()` and `ir()`. Run it on the cross-sectional BBIBOLL factor from Chapter 8. Report the mean IC, IR, and t-statistic (is $|t| > 2$?).

9.4 Experiment: BBIBOLL Factor Evaluation

Experiment 9.1. **Goal**: Evaluate the BBIBOLL deviation signal as a cross-sectional factor.

Method:

1. Compute the cross-sectional z-scored BBIBOLL deviation factor for the liquid universe.
2. Compute the 5-day forward return for each stock at each date.
3. Run `FactorAnalyzer`: compute IC time series, mean IC, IR, t-stat.
4. Plot the IC decay curve for horizons 1, 2, 5, 10, 20 days.
5. Compute daily turnover.

Questions to answer:

- Is the mean IC positive or negative? (Directional prediction.)
- Is $|t| > 2$? (Statistically significant?)
- What is the natural horizon? (Where IC peaks before decaying.)
- Is turnover manageable? (Can you trade this weekly?)

Reflection space: *(fill in after running)*

Chapter 10

Factor Construction

10.1 Motivation

A raw signal is rarely optimal. Cross-sectional normalization, sector neutralization, and outlier handling can substantially improve IC and IR — or reveal that the signal was just a proxy for a well-known risk factor.

This chapter covers the standard transformations every factor goes through before it is ready for combination and portfolio construction.

10.2 Concepts

10.2.1 Normalization

Definition 10.1 (Z-Score Normalization)**.**

$$z_{i,t} = \frac{x_{i,t} - \bar{x}_t}{\sigma_{x,t}}$$

Applied cross-sectionally: ensures zero mean, unit variance across the stock universe at each date.

Definition 10.2 (Rank Normalization)**.** Replace raw values with their cross-sectional rank, then map to $[-1, 1]$ or to a standard normal via the inverse CDF. More robust to outliers than z-scoring.

Definition 10.3 (Winsorization)**.** Cap extreme values at a percentile threshold (e.g., 1st and 99th percentiles). Applied *before* z-scoring to prevent outliers from dominating the mean and standard deviation.

10.2.2 Neutralization

Definition 10.4 (Sector Neutralization)**.** Demean the factor within each sector:

$$z_{i,t}^{\text{neutral}} = z_{i,t} - \bar{z}_{\text{sector}(i),t}$$

Purpose: remove sector-level effects so the factor captures stock-specific alpha, not sector rotation.

Definition 10.5 (Size and Beta Neutralization)**.** Regress the factor on log-market-cap and/or market beta; use the residual:

$$z^{\perp} = z - \hat{z}$$

where $\hat{z}$ is the fitted value from the regression. This isolates alpha from well-known risk premiums.

10.2.3 Orthogonalization

Definition 10.6 (Factor Orthogonalization)**.** Given existing factors $F_1, \ldots, F_k$, orthogonalize a new factor F_{k+1} by regressing it on the existing set and taking the residual:

$$F_{k+1}^{\perp} = F_{k+1} - X(X^T X)^{-1} X^T F_{k+1}$$

where $X = [F_1 \mid \cdots \mid F_k]$.

The residual $F_{k+1}^{\perp}$ contains only the information in F_{k+1} that is **not** already captured by the existing factors.

Remark 10.1. Orthogonalization order matters. F_2 orthogonalized against F_1 is different from F_1 orthogonalized against F_2. The first factor in the sequence "wins" more of the shared variance.

10.3 Implementation Sketch

A factor preprocessing pipeline:

```python
def preprocess_factor(
    raw_signal: pl.DataFrame,        # (date, ticker, value)
    sectors: pl.DataFrame,           # (ticker, sector)
    market_cap: pl.DataFrame,        # (date, ticker, log_mcap)
    winsorize_pct: float = 0.01,
    neutralize: list[str] = ["sector"],
) -> pl.DataFrame:
    """
    1. Winsorize at (pct, 1-pct) cross-sectionally.
    2. Z-score cross-sectionally.
    3. Neutralize for specified factors (sector, size).
    4. Return the preprocessed factor DataFrame.
    """
```

Lab Module 10.1. Build it: Implement `preprocess_factor()` with at least z-scoring and sector neutralization. Compare the IC of the raw BBIBOLL factor vs. the sector-neutralized version. Does neutralization improve IR?

10.4 Experiment: Neutralization Impact

Experiment 10.1. **Goal**: Measure how sector neutralization affects the BBIBOLL factor.

Method:

1. Compute the raw z-scored BBIBOLL factor.
2. Apply sector neutralization using GICS sector assignments.
3. Compute IC, IR, and turnover for both versions.
4. Compare.

Hypothesis: If BBIBOLL is mostly capturing sector rotation (e.g., "tech is cheap relative to its own history"), neutralization will *destroy* the IC. If it captures stock-specific mean reversion, neutralization will *preserve or improve* it.

Reflection space: *(fill in after running)*

Chapter 11

Factor Combination

11.1 Motivation

No single factor is strong enough to build a portfolio on. The power of systematic investing comes from **combining** many weak signals into a composite that is more stable than any individual factor.

This chapter covers the main combination methods, from the simplest (equal-weight) to the most sophisticated (mean-variance on IC covariance).

11.2 Concepts

11.2.1 Equal-Weight and IC-Weight

The simplest composite: $\text{composite}_{i,t} = \sum_k w_k \cdot f_{k,i,t}$ where f_k are z-scored factors.

- **Equal-weight**: $w_k = 1/K$ for all factors. No estimation risk, but ignores differences in signal quality.
- **IC-weight**: $w_k \propto \overline{IC}_k$. Allocates more weight to stronger factors. Requires reliable IC estimates.

11.2.2 Mean-Variance on Factor ICs

Definition 11.1 (Factor Mean-Variance Combination)**.** Treat each factor's IC time series as a "return stream." Apply mean-variance optimization:

$$w^* = \arg\max_w \left(w^T \mu_{IC} - \frac{\lambda}{2} w^T \Sigma_{IC}\, w \right)$$

where μ_{IC} is the vector of mean ICs and Σ_{IC} is the covariance matrix of IC time series.

This accounts for both factor quality (mean IC) and factor correlation (diversification benefit from uncorrelated factors).

11.2.3 Risk Parity

Definition 11.2 (Risk Parity Combination)**.** Weight factors so that each contributes equally to the composite signal's variance:

$$w_k \propto \frac{1}{\sigma_{IC,k}}$$

Advantage: does not require estimating expected IC (only volatility), so it is more robust to estimation error.

Remark 11.1. In practice, start with equal-weight. Only graduate to IC-weight or mean-variance when you have enough factors (≥ 5) and enough history (≥ 2 years of daily IC) to estimate the inputs reliably.

Lab Module 11.1. Build it: Once you have ≥ 2 factors (e.g., BBIBOLL deviation + 20-day momentum), implement equal-weight and IC-weight combination. Compare the composite's IC and IR against each individual factor. Does combination improve IR?

11.3 Experiment: Two-Factor Composite

Experiment 11.1. **Goal**: Build and evaluate a two-factor composite signal.

Method:

1. Factor A: Cross-sectional z-scored BBIBOLL deviation.
2. Factor B: 20-day momentum (cross-sectional z-scored).
3. Compute the Spearman correlation between A and B (across stocks, averaged over time). Are they diversifying?
4. Form composites: equal-weight and IC-weight.
5. Evaluate each with `FactorAnalyzer`: IC, IR, decay.

Expected: If A and B are negatively correlated (mean-reversion vs. momentum), the composite's IR should be higher than either alone.

Reflection space: *(fill in after running)*

Part IV

Portfolio, Risk, and Execution

A signal is not a portfolio. Turning alpha signals into actual positions requires risk management, position sizing, and realistic execution cost modeling. This part covers the bridge from "I have a good factor" to "I have a tradeable strategy."

Chapter 12

Risk Measures

12.1 Motivation

Risk management answers the question: "how bad can it get?" Without it, a strategy with positive expected return can still bankrupt you via a single tail event.

Interview relevance: "What risk measures do you use?" and "Explain the difference between VaR and CVaR" are standard questions.

12.2 Concepts

12.2.1 Value at Risk

Definition 12.1 (VaR)**.** The maximum loss over a given time horizon at a given confidence level:

$$P(L > \mathrm{VaR}_\alpha) = 1 - \alpha$$

Example: "1-day 95% VaR = \$10,000" means there is a 5% chance of losing more than \$10,000 in one day.

Three estimation methods:

- **Historical**: Use the empirical return distribution.
- **Parametric**: Assume a distribution (e.g., Gaussian) and compute the quantile analytically.
- **Monte Carlo**: Simulate many return paths and compute the empirical quantile.

Definition 12.2 (CVaR (Expected Shortfall))**.**

$$\mathrm{CVaR}_\alpha = \mathbb{E}[L \mid L > \mathrm{VaR}_\alpha]$$

The expected loss *given that* the loss exceeds VaR. CVaR captures tail risk better than VaR because it asks "when things go bad, *how* bad?"

12.2.2 Distribution Properties of Returns

Definition 12.3 (Skewness and Kurtosis)**.**

- **Skewness**: $\gamma_1 = \mathbb{E}\left[\left(\frac{X-\mu}{\sigma}\right)^3\right]$. Negative skew = more extreme losses than gains.
- **Excess Kurtosis**: $\kappa = \mathbb{E}\left[\left(\frac{X-\mu}{\sigma}\right)^4\right] - 3$. Positive κ = fat tails.

Equity returns typically have **negative skewness** and **positive excess kurtosis**. Gaussian VaR underestimates tail risk.

Lab Module 12.1. Build it: Compute the daily return distribution of the BBIBOLL strategy. Plot the histogram against a fitted Gaussian. Compute skewness, excess kurtosis, 95% VaR (historical and parametric), and CVaR. How much does the parametric VaR underestimate tail risk?

Chapter 13

Portfolio Construction

13.1 Motivation

A factor tells you *which* stocks to favor. Portfolio construction tells you *how much* of each stock to hold. The mapping from signal to position is where risk management and alpha research meet.

13.2 Concepts

13.2.1 Factor Exposure and CAPM

Definition 13.1 (Alpha and Beta)**.** From the CAPM regression: $r_i - r_f = \alpha_i + \beta_i(r_m - r_f) + \epsilon_i$

- β_i: sensitivity to the market. $\beta = 1$ means the stock moves with the market.
- α_i: excess return not explained by market movement. **This is what we seek.**

Definition 13.2 (Market-Neutral Portfolio)**.** A portfolio with net $\beta \approx 0$: long stocks with positive alpha, short stocks with negative alpha, weighted so that the net market exposure cancels. Returns are (approximately) independent of market direction.

13.2.2 Position Sizing

Definition 13.3 (Equal-Weight)**.** Allocate $1/N$ of capital to each position. Simple, no estimation risk, but ignores differences in signal strength and risk.

Definition 13.4 (Volatility Targeting)**.** Scale position sizes inversely with each stock's recent volatility:

$$w_i \propto \frac{\sigma_{\text{target}}}{\hat{\sigma}_i}$$

This ensures each position contributes roughly equal risk to the portfolio.

Definition 13.5 (Kelly Criterion)**.** For a portfolio with expected excess return vector μ and covariance matrix Σ:

$$f^* = \Sigma^{-1}\mu$$

Full Kelly maximizes long-run growth but has extreme drawdowns. In practice, **half-Kelly** ($f^*/2$) or even quarter-Kelly is used.

Lab Module 13.1. Build it: Implement volatility-targeted position sizing. Compare equal-weight vs. vol-targeted performance for the BBIBOLL strategy. Measure: Sharpe, MDD, Calmar.

Chapter 14

Execution and Transaction Costs

14.1 Motivation

A backtest without transaction costs is a fantasy. The gap between "paper alpha" and "live alpha" is almost entirely explained by execution costs: commissions, bid-ask spread, slippage, and market impact.

This chapter covers the mechanics of execution and how to model costs in a backtest.

Interview relevance: "How do you model transaction costs in your backtest?" is a question that separates candidates who have thought about real trading from those who haven't.

14.2 Concepts

14.2.1 Cost Components

Definition 14.1 (Bid-Ask Spread)**.** Spread $= P_{\text{ask}} - P_{\text{bid}}$. Crossing the spread to execute a market order costs approximately half the spread per side.

Definition 14.2 (Slippage)**.** The difference between the expected execution price and the actual fill. Caused by: order book depth, timing delays, stale quotes.

Definition 14.3 (Market Impact)**.** The price movement caused by your own order:

- **Temporary impact**: Price displacement during execution; reverts afterward.
- **Permanent impact**: Information leakage that shifts the fair price.

Square-root model (empirical): Impact $\approx \sigma \cdot \sqrt{V_{\text{order}}/V_{\text{daily}}}$

14.2.2 Execution Algorithms

Definition 14.4 (VWAP and TWAP)**.** Algorithms that slice large orders to reduce impact:

- **VWAP**: Execute proportional to historical intraday volume profile.
- **TWAP**: Execute equally over fixed time intervals.

14.2.3 Cost Modeling in Backtests

Three levels of sophistication:

1. **Fixed cost**: Deduct a fixed amount per trade (e.g., \$0.005/share). Simple but unrealistic for large or illiquid orders.

2. **Spread model**: Deduct half the typical bid-ask spread per side. Requires spread data (or estimates from Amihud illiquidity).

3. **Impact model**: Use the square-root model above. Requires volume data. Most realistic, but adds complexity.

Common Pitfall 14.1. Ignoring costs is the most common source of "paper alpha" that disappears in live trading. Even a simple fixed-cost model is infinitely better than zero-cost.

Lab Module 14.1. Build it: Add a transaction cost model to the backtest engine. Start with level 1 (fixed cost), then upgrade to level 2 (spread model). Re-run the BBIBOLL backtest with costs. How much does the Sharpe drop? At what cost level does the strategy become unprofitable?

14.3 Experiment: Cost Sensitivity Analysis

Experiment 14.1. **Goal**: Determine how sensitive the BBIBOLL strategy's performance is to transaction costs.

Method:

1. Run the backtest with costs at 0, 1, 5, 10, 20 bps per side.

2. Plot Sharpe ratio as a function of cost.

3. Find the breakeven cost (where Sharpe $= 0$).

Interpretation: If the breakeven cost is < 5 bps, the strategy is probably not tradeable in practice. If > 20 bps, it is robust.

Reflection space: *(fill in after running)*

Part V

Machine Learning in Quantitative Finance

Machine learning in quant finance is not about model complexity. It is about ***feature engineering****,* ***proper validation****, and* ***respecting the time-series structure*** *of financial data. If you get these three things right, a simple model will beat a complex one.*

Chapter 15

Feature Engineering for Finance

15.1 Motivation

The gap between "ML engineer" and "quant ML researcher" is not modeling — it is features. A LightGBM model with 50 carefully engineered financial features will outperform a deep neural network fed raw OHLCV.

15.2 Concepts

15.2.1 Feature Categories

Definition 15.1 (Financial Feature Taxonomy)**.**

- **Momentum**: Returns over multiple horizons (1d, 5d, 20d, 60d, 252d).
- **Mean reversion**: Distance from moving averages (e.g., the BBIBOLL deviation signal).
- **Volume**: Relative volume, OBV, volume-price divergence, Amihud illiquidity.
- **Volatility**: Realized vol (from intraday data), GARCH residuals, vol-of-vol.
- **Fundamental**: PE, PB, earnings yield, dividend yield, revenue growth (requires fundamental data).
- **Microstructure**: Bid-ask spread statistics, trade imbalance, Kyle's lambda (requires tick data).

15.2.2 Feature Processing

- **Cross-sectional normalization**: Z-score or rank-transform features across stocks at each date (same as factor preprocessing in Chapter 10).
- **Lag features**: Use the signal value from $t-1$, $t-2$, etc. as additional inputs. Captures mean-reversion or momentum in the signal itself.
- **Interaction features**: Ratios or products of base features (e.g., momentum $\times$ volume = "confirmed momentum").

Common Pitfall 15.1. **Leakage through normalization.** If you z-score using the full sample mean/std (including future dates), you introduce look-ahead bias. Always normalize using only data available up to date t.

Lab Module 15.1. Build it: Construct a feature matrix with ≥ 10 features (momentum at 3 horizons, BBIBOLL deviation, OBV z-score, realized vol, etc.) for the liquid universe. Inspect the cross-sectional distribution of each feature — any extreme outliers? Apply winsorization and z-scoring.

Chapter 16

Time-Series Validation

16.1 Motivation

Standard k-fold cross-validation is **invalid** for time series because it uses future data to predict the past. This is the single most common ML mistake in quant finance — and the easiest way to produce a backtest that looks incredible on paper and fails immediately in live trading.

16.2 Concepts

16.2.1 Validation Schemes

Definition 16.1 (TimeSeriesSplit)**.**

- **Expanding window**: Train on $[0, t]$, test on $[t + 1, t + h]$. Increase t each iteration.
- **Rolling window**: Train on $[t - w, t]$, test on $[t + 1, t + h]$. Fixed training window width w.
- **Purged / Embargoed**: Remove g days of data around the train/test boundary to prevent label leakage from overlapping forward-return windows.

Definition 16.2 (Walk-Forward Training)**.** The ML equivalent of walk-forward backtesting (Chapter 7): retrain the model periodically (e.g., monthly) on the most recent N months, predict the next month, concatenate all out-of-sample predictions.

Common Pitfall 16.1. **Hyperparameter tuning on the test set.** If you tune `max_depth` or `learning_rate` by looking at test-set performance, you are overfitting to the test set. Use a nested validation scheme: train / validation / test.

Lab Module 16.1. Build it: Implement a `PurgedWalkForwardCV` class that yields (train, test) index pairs with a configurable embargo gap. Verify that no test date appears in any training fold.

Chapter 17

Tree Models and Ensembles

17.1 Motivation

In quant ML, **gradient-boosted trees** (LightGBM, XGBoost) are the workhorse. They handle tabular data naturally, capture nonlinear interactions, and provide built-in feature importance. A well-tuned LightGBM with proper walk-forward validation is a strong baseline that is hard to beat with more complex models.

17.2 Concepts

17.2.1 Why Trees for Finance?

- **Tabular data**: Financial features are structured columns, not images or text. Trees excel on tabular data.
- **Nonlinear interactions**: The relationship between momentum and future return may depend on volatility. Trees capture this automatically via splits.
- **Feature importance**: Built-in gain/split importance helps with feature selection and factor discovery.
- **Speed**: LightGBM trains on 10^6 rows $\times$ 50 features in seconds.

17.2.2 Ensemble Methods

Definition 17.1 (Bagging vs. Boosting).

- **Bagging** (Random Forest): Train many trees on bootstrap samples; average predictions. Reduces *variance*.
- **Boosting** (LightGBM, XGBoost): Train trees sequentially, each correcting the previous one's errors. Reduces *bias*.

In practice, boosting dominates for tabular prediction tasks.

17.2.3 Hyperparameter Discipline

Key hyperparameters for LightGBM in a quant context:

- `max_depth`: Limit to 4–6 to prevent overfitting to noise.
- `num_leaves`: Control tree complexity (related to depth).
- `min_data_in_leaf`: At least 100–500 for noisy financial data.
- `learning_rate`: Small (0.01–0.05) with many boosting rounds and early stopping.

- `feature_fraction`: Subsample features per tree (0.5–0.8) for regularization.

Remark 17.1. The project already includes LightGBM as a dependency, but no ML-based strategy has been implemented yet. The recommended sequence: build the factor evaluation framework (Part III) first, then use ML to *combine* factors, not to *replace* them.

Lab Module 17.1. Build it: Train a LightGBM model to predict 5-day forward return *direction* (binary classification) using the feature matrix from Chapter 15. Use `PurgedWalkForwardCV` from Chapter 16.

1. Train on 12 months, predict the next month. Slide monthly.
2. Report out-of-sample AUC and accuracy.
3. Extract feature importance. Which features matter most?
4. Compare with a simple equal-weight factor composite (Part III). Does ML add incremental alpha?

17.3 Experiment: LightGBM vs. Linear Factor Model

Experiment 17.1. **Goal**: Determine whether ML adds value beyond a linear factor composite.
Method:

1. Baseline: Equal-weight composite of 5+ factors (Part III result). Evaluate with IC/IR.
2. LightGBM: Same features, walk-forward trained. Convert predicted probabilities to a cross-sectional signal. Evaluate with IC/IR.
3. Compare IC, IR, and portfolio-level Sharpe.

Hypothesis: ML will capture nonlinear interactions and improve IC by 10–30%, but at the cost of lower interpretability and higher turnover.
Reflection space: *(fill in after running)*

Appendix A

Mathematical Reference

This appendix collects the mathematical foundations referenced by the main chapters. It is not meant to be read front-to-back — flip here when a chapter says "see Appendix A."

A.1 Probability and Statistics

Definition A.1 (Random Variable)**.** A measurable function $X : \Omega \to \mathbb{R}$ from a sample space to the reals. In finance, X typically represents a future return or a signal value.

Definition A.2 (Expected Value)**.** Discrete: $\mathbb{E}[X] = \sum_x x \cdot p(x)$. Continuous: $\mathbb{E}[X] = \int x\, f(x)\, dx$.

Definition A.3 (Variance and Standard Deviation)**.**

$$\mathrm{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2] \qquad \sigma(X) = \sqrt{\mathrm{Var}(X)}$$

In finance, σ of returns is the standard measure of **volatility**.

Definition A.4 (Covariance and Correlation)**.**

$$\mathrm{Cov}(X, Y) = \mathbb{E}[(X - \mu_X)(Y - \mu_Y)] \qquad \rho(X, Y) = \frac{\mathrm{Cov}(X, Y)}{\sigma_X \sigma_Y}$$

Correlation measures *linear* dependence only.

Definition A.5 (Conditional Expectation)**.** $\mathbb{E}[X \mid Y = y]$: the expected value of X given that $Y = y$. This is the foundation of alpha research — "what is the expected return conditional on observing this signal value?"

A.1.1 Key Theorems

Definition A.6 (Law of Large Numbers)**.** As $n \to \infty$: $\bar{X}_n \to \mathbb{E}[X]$. *Practical*: Your backtest's average return converges to the true expected return — if the process is stationary.

Definition A.7 (Central Limit Theorem)**.** For i.i.d. X_i with mean μ, variance σ^2:

$$\frac{\bar{X}_n - \mu}{\sigma/\sqrt{n}} \xrightarrow{d} \mathcal{N}(0, 1)$$

Practical: Justifies t-tests on Sharpe ratios and IC values — but financial returns violate i.i.d. (autocorrelation, fat tails), so apply with caution.

A.1.2 Hypothesis Testing

Definition A.8 (Hypothesis Test)**.** Test H_0 (e.g., "IC = 0") against H_1 ("IC $\neq$ 0"):

1. Compute a test statistic (e.g., $t = \bar{x}/(s/\sqrt{n})$).
2. Find the p-value: probability of observing this extreme a value under H_0.
3. Reject H_0 if $p < \alpha$ (typically $\alpha = 0.05$).

A.1.3 Regression

Definition A.9 (OLS)**.** Minimize $\sum_i (y_i - x_i^T \beta)^2$. Solution: $\hat{\beta} = (X^T X)^{-1} X^T y$.

In quant finance: cross-sectional regression of stock returns on factor exposures yields **factor returns** at each date.

Definition A.10 (Regularization)**.** When factors are correlated:

- **Ridge (L2)**: $+\lambda \|\beta\|_2^2$. Shrinks coefficients.
- **Lasso (L1)**: $+\lambda \|\beta\|_1$. Drives irrelevant coefficients to zero (feature selection).

A.2 Time Series

Definition A.11 (Stationarity)**.** A time series $\{X_t\}$ is weakly stationary if $\mathbb{E}[X_t] = \mu$ for all t and $\mathrm{Cov}(X_t, X_{t+h})$ depends only on the lag h. Prices are non-stationary; returns are approximately stationary.

Definition A.12 (Unit Root and ADF Test)**.** Unit root: $X_t = X_{t-1} + \epsilon_t$ (random walk). The Augmented Dickey–Fuller test tests H_0: unit root exists. Reject $\Rightarrow$ the series is stationary.

Definition A.13 (Cointegration)**.** Two non-stationary series X_t, Y_t are cointegrated if $\exists\, \beta$ such that $Y_t - \beta X_t$ is stationary. Foundation of pairs trading / statistical arbitrage.

Definition A.14 (ACF / PACF)**.**

- ACF: $\rho(h) = \mathrm{Corr}(X_t, X_{t+h})$.
- PACF: Correlation between X_t and X_{t+h} after removing linear effects of intermediate lags. Diagnostic tools for identifying AR and MA order.

Definition A.15 (ARIMA)**.** AR(p): $X_t = \sum_{i=1}^{p} \phi_i X_{t-i} + \epsilon_t$. MA($q$): $X_t = \epsilon_t + \sum_{j=1}^{q} \theta_j \epsilon_{t-j}$. ARIMA($p, d, q$): ARMA on the d-th difference.

Definition A.16 (GARCH(1,1))**.**

$$\sigma_t^2 = \omega + \alpha\, \epsilon_{t-1}^2 + \beta\, \sigma_{t-1}^2$$

Captures volatility clustering: today's variance depends on yesterday's shock and variance.

Definition A.17 (Realized Volatility)**.** $RV_t = \sum_{i=1}^{N} r_{t,i}^2$ where $r_{t,i}$ are intraday returns (e.g., 5-minute). Uses high-frequency data to estimate daily vol more accurately than daily close-to-close.

A.3 Stochastic Calculus

These are foundations for derivatives pricing and advanced risk modeling. Not immediately needed for equity alpha research, but essential for a complete quant education.

Definition A.18 (Brownian Motion)**.** A continuous-time process W_t with $W_0 = 0$, independent Gaussian increments ($W_t - W_s \sim \mathcal{N}(0, t-s)$), and continuous paths.

Definition A.19 (Geometric Brownian Motion)**.**

$$dS_t = \mu S_t \, dt + \sigma S_t \, dW_t$$

Solution: $S_t = S_0 \exp((\mu - \sigma^2/2)t + \sigma W_t)$. The basis of the Black–Scholes model.

Definition A.20 (Itô's Lemma)**.** For $f(S_t, t)$ where S_t follows an Itô process:

$$df = \frac{\partial f}{\partial t} \, dt + \frac{\partial f}{\partial S} \, dS + \frac{1}{2} \frac{\partial^2 f}{\partial S^2} (dS)^2$$

The $(dS)^2$ term distinguishes stochastic calculus from ordinary calculus.

Definition A.21 (Martingale)**.** $\{M_t\}$ is a martingale if $\mathbb{E}[M_t \mid \mathcal{F}_s] = M_s$ for all $s \leq t$. Under the risk-neutral measure, discounted asset prices are martingales.

Appendix B

Quant Notes

This appendix records key insights discovered during development and study. Each entry captures the date, the question that arose, the finding, and why it matters. Entries are ordered chronologically.

Entry 1 — Individual Factor IR Is Weak by Design

Date 2026-03-02 (from `alpha_iteration.ipynb`)

Question
All four factors (BBIBOLL, STR, VPD, Vol Ratio) showed |IR| in the 0.05–0.12 range. Are these signals too weak to be useful?

Finding No — this is *normal*. In practice, single-factor IRs rarely exceed 0.3; values above 0.5 are exceptional. The practical ranges are roughly:

- $|\text{IR}| < 0.05$: noise, not tradeable.
- $0.05 \leq |\text{IR}| < 0.15$: weak but real — the typical range for individual alpha factors.
- $0.15 \leq |\text{IR}| < 0.30$: solid single factor.
- $|\text{IR}| \geq 0.50$: suspicious — likely overfitted or measured in-sample.

Why It Matters
The **Fundamental Law of Active Management** (Grinold & Kahn) explains how weak signals become useful:

$$\text{IR}_{\text{portfolio}} \approx \text{IR}_{\text{single}} \times \sqrt{N}$$

where N is the number of *independent* bets (signals × rebalance periods). Combining 10 orthogonal factors each with IR = 0.10 gives a portfolio-level IR $\approx 0.10 \times \sqrt{10} \approx 0.32$, well into the "strong" range. The game is not finding one brilliant factor; it is combining many weak-but-orthogonal signals.

Entry 2 — Anti-Correlation Requires Aligned IC Signs

Date 2026-03-02 (from `alpha_iteration.ipynb`)

Question

BBIBOLL and STR have strongly anti-correlated daily IC ($\rho = -0.80$). Anti-correlation should be great for diversification — why did equal-weight combination *dilute* the signal instead?

Finding Anti-correlation in the IC *series* means the two factors "take turns being right." But whether that helps depends on whether they agree on **direction**:

- **Same IC sign + anti-correlated → great.** Both factors predict the same direction (e.g. both negative IC). On days factor A is strong, factor B is weak — but they still point the same way. The composite mean IC is preserved while the variance drops, so IR rises.
- **Opposite IC sign + anti-correlated → cancellation.** BBIBOLL has IC $= -0.022$ and STR has IC $= +0.013$. When BBIBOLL fires strongly (IC ≈ -0.05), the anti-correlation means STR also fires strongly — but in the *opposite* direction (IC $\approx +0.04$). Equal-weight averaging: $(-0.05 + 0.04)/2 \approx -0.005$. The signals cancel, collapsing the mean IC.

The Math

For two equal-weight factors with IC means μ_A, μ_B, IC standard deviations σ_A, σ_B, and IC correlation ρ:

$$\mu_{\text{combo}} = \tfrac{1}{2}(\mu_A + \mu_B)$$
$$\sigma^2_{\text{combo}} = \tfrac{1}{4}(\sigma_A^2 + \sigma_B^2 + 2\rho\,\sigma_A\sigma_B)$$

Anti-correlation ($\rho < 0$) always shrinks σ^2_{combo} (the denominator of IR). But if μ_A and μ_B have opposite signs, μ_{combo} also shrinks (the numerator of IR). The net effect on IR $= \mu/\sigma$ depends on which shrinks faster. In the BBIBOLL × STR case, the numerator collapsed more than the denominator — so IR fell.

Experimental Evidence

- BBIBOLL + Vol Ratio ($\rho = 0.02$, same IC sign): composite $|\text{IR}| = 0.136 > 0.122$ (best individual). **Diversification confirmed.**
- BBIBOLL + STR ($\rho = -0.80$, opposite IC sign): composite IR diluted. Anti-correlation *hurt*.

Why It Matters

The analogy is portfolio construction: two negatively correlated assets reduce portfolio volatility only if you are **long both**. If you are long one and short the other, negative correlation means they move together — more risk, not less. Similarly, combining factors with opposite IC signs is like going long one alpha and short the other; the anti-correlation amplifies cancellation rather than providing stability.

Interview one-liner: *"Anti-correlated ICs reduce tracking error, but only improve IR if the factors agree on direction. Opposite IC signs turn anti-correlation from a hedge into a cancellation."*

Entry 3 — Strategy-Based vs. Alpha-Pipeline Backtesting

Date 2026-03-02 (architecture comparison)

Question

The codebase offers two backtest paths:

- `BBIBOLLStrategy` → `BacktestEngine` (in `src/strategy/`)
- `pipeline.py` → `WeightBacktester` (in `src/portfolio/`)

What is the fundamental difference, and when should each be used?

Finding They embody two distinct mental models of systematic investing.

Strategy approach — "I Trade."

1. Calculate indicators (BBI + Bollinger Bands) per ticker.
2. Generate signals by iterating *row by row*, tracking stateful holding logic via `is_holding` and `buy_price`. Emit buy(+1)/sell(−1) on threshold triggers.
3. Match buy→sell pairs via `join_asof`; compute per-trade P&L.
4. Expand trades into daily holdings; equal-weight average → equity curve.

Key properties: *stateful* (sell decision depends on entry price), *discrete events* (in or out), *single-asset logic* (each ticker processed independently), *trade-centric output* ("bought AAPL at $150, sold at $165, return = +10%").

Pipeline approach — "I Allocate."

1. Compute factors (bbiboll deviation, volatility ratio, momentum) for *all* stocks simultaneously.
2. Rank cross-sectionally on each rebalance date; assign portfolio weights (top-ranked → equal weight, rest → zero).
3. Portfolio return: $r_t = \sum_i w_{i,t}\, r_{i,t}$ — no explicit trades, just weights × returns.

Key properties: *stateless* (today's weight depends only on today's factor values), *continuous allocation* (how much, not whether), *cross-sectional* (all stocks compete for capital), *portfolio-centric output* (daily portfolio return, Sharpe ratio).

Summary Table

Dimension	Strategy	Pipeline
Unit of analysis	Individual trade	Portfolio weight
State	Entry price, holding flag	None (recomputed fresh)
Decision	Buy/sell *this* stock?	Allocate across *all* stocks?
Output	Trade list + P&L	Weight matrix + return series
Rebalance	Signal-triggered	Calendar-based (weekly/monthly)
Typical role	Trader / execution	Researcher / PM

Why It Matters

Most quant interview questions and institutional workflows assume the **pipeline worldview**: "describe an alpha factor", "construct a long-short portfolio", "what is your Sharpe?" The strategy approach is valuable for execution-level or high-frequency work, but the pipeline is closer to how firms like Citadel, Two Sigma, and AQR structure research.

The bridge between the two is `WeightBacktester`: it accepts pipeline output (weights) and produces the equity curve + Sharpe that connect the "I allocate" mindset to concrete performance metrics. In the codebase, `backtester.py -mode strategy` runs the trade-level path and `-mode pipeline` runs the portfolio-level path, allowing direct comparison on the same data.

Learning path: start with the strategy approach (it is intuitive — you can inspect individual trades), then graduate to the pipeline (it forces you to think in portfolios:

factors → ranking → allocation → rebalancing). The key insight from the **Fundamental Law of Active Management** (Entry B) is that the pipeline naturally maximises the number of independent bets N, which is how weak individual signals become a strong portfolio.

Entry 4 — Kelly Criterion Is a Leverage Tool, Not an Allocation Tool

Date 2026-03-02 (from `pipeline_demo.ipynb` ablation study)

Question

We implemented "Half-Kelly" position sizing: $w_i \propto \mu_i/\sigma_i^2$, normalized to unit gross leverage. Every configuration using it produced *worse* Sharpe than equal-weight. Three successive bug fixes failed to improve it. What is fundamentally wrong?

Background — The Kelly Formula

The Kelly criterion solves: *"What fraction of my bankroll should I wager on a bet with known edge μ and variance σ^2?"*

$$f^* = \frac{\mu}{\sigma^2}$$

This is optimal for a **single repeated bet** with independent outcomes, maximizing the long-run geometric growth rate $G = \mathbb{E}[\log(1 + f \cdot r)]$.

Half-Kelly ($f^*/2$) is a common practitioner adjustment that sacrifices roughly 25% of growth for substantially lower drawdown. The key property: *Kelly tells you how much total capital to risk*, not how to split capital across multiple simultaneous bets.

The Three Bugs We Hit (and Fixed)

1. **Normalization destroyed the leverage signal.** We normalized $\sum |w_i| = 1$, which discards Kelly's answer to "how much to bet." After normalization, only the *relative* sizing remains — but that was never Kelly's purpose. *Fix:* let leverage float (partly undone by needing comparable backtest results).
2. **Using μ_i from returns made Kelly ignore the factor.** Historical mean return over 60 days is dominated by noise (SNR ≈ 0.02); the factor signal was unused. All `AlphaConfig` variants produced identical Sharpe = 0.343 regardless of factor choice. *Fix:* two-stage design — factor selects stocks, Kelly sizes by $|\mu_i|/\sigma_i^2$.
3. **Historical μ is stealth momentum.** Even after the two-stage fix, μ_i from a 60-day window injects a momentum bias. For a mean-reversion signal like BBIBOLL, this conflicts with the factor direction. *Fix:* replace μ_i with z_i, the cross-sectional z-score of the factor signal itself.

The Ablation Study

After all three fixes, we ran a 2×2 experiment isolating sizing and rebalancing frequency:

		Rebalance	
		Daily	Weekly
Sizing	Equal-Weight	+0.814	+0.417
	Signal-Weighted	+0.137	−0.393

Two independent sources of Sharpe degradation:

- **Weekly rebalancing** $\rightarrow -0.40$ Sharpe (mean-reversion alpha decays intraweek).
- **Signal-weighted sizing** $\rightarrow -0.68$ Sharpe (concentration penalty).

Root Cause — Why "Kelly Sizing" Hurts

The z_i/σ_i^2 formula concentrates weight into low-volatility, high-conviction names. But for BBIBOLL (a mean-reversion signal), the strongest alpha comes from *high-volatility* stocks that have deviated far from their bands. Signal-weighted sizing systematically **under-weights the best opportunities**.

Mathematically: $w_i \propto z_i/\sigma_i^2$ penalizes large σ_i quadratically. In cross-section, the stocks with the largest $|z_i|$ (strongest signal) also tend to have the largest σ_i (that is what drove the deviation). The penalty overwhelms the conviction signal.

The Conceptual Error

Kelly solves a fundamentally different problem than what we asked it to do:

	Kelly's Actual Problem	**What We Used It For**
Question	How much total capital to deploy?	How to split capital across N stocks?
Input	Edge and variance of *one* bet	Cross-sectional factor signal
Output	Portfolio-level leverage f^*	Relative stock weights w_i
Regime	Independent repeated bets	Correlated concurrent positions

In a multi-asset portfolio, the correct Kelly generalization is the **multivariate Kelly**:

$$\mathbf{f}^* = \Sigma^{-1}\boldsymbol{\mu}$$

where Σ is the covariance matrix and $\boldsymbol{\mu}$ is the expected return vector. This accounts for cross-correlations and produces a single leverage-scaled portfolio. Our per-stock μ_i/σ_i^2 ignores correlations entirely.

Resolution

1. **Renamed** `size_half_kelly` $\rightarrow$ `size_signal_weighted` with docstring explicitly stating "Not true Kelly."
2. **Formula kept** as $w_i \propto \text{direction}_i \times |z_i|/\sigma_i^2$ — it is a valid conviction $\times$ inverse-variance sizing method, just not Kelly.
3. **True Kelly deferred** to a future portfolio-level leverage module that would answer: "given my composite alpha signal, should I deploy $0.5\times$ or $1.5\times$ capital today?"

Why It Matters

1. **Naming matters.** Calling something "Kelly" imports assumptions (independent bets, known μ) that do not hold. Honest names prevent honest mistakes.
2. **Position sizing is not "just implementation."** With the *same* signal, Sharpe ranged from -0.39 to $+0.81$ — a 1.2 swing. Sizing is a core part of the strategy, not a post-hoc detail.
3. **Kelly requires reliable μ estimates.** With $|\text{IC}| \approx 0.02$, the expected return signal is noise. Kelly amplifies noise. It becomes useful only when $\text{IC} > 0.05$ and preferably with multivariate correlations.

Interview one-liner: *"Kelly tells you how much to bet, not how to split the bet. Using scalar Kelly for cross-sectional allocation ignores correlations and concentrates into low-vol names — exactly the wrong thing for a volatility-driven alpha."*

Entry 5 — Position Sizing Is Portfolio Construction, Not Just Weighting

Date 2026-03-04

Context After fixing the forced-short bug (Entry 5b below) and renaming Half-Kelly → Signal-Weighted (Entry 4), we stepped back to examine the position sizing module as a whole. All four methods share the same two-stage architecture:

1. **Selection** — rank stocks by signal, pick top-N long and bottom-N short (binary in/out).
2. **Sizing** — within each leg, assign weights (equal, inverse-vol, vol-target, or signal-weighted).

This is the standard *rank-and-select* approach from academic factor research (Fama–French quintile portfolios).

The Forced-Short Problem

Setting `n_short=10` forces the portfolio to short 10 stocks even when the signal says they are merely *weaker longs*, not actual short candidates. Our alpha signal is directional (higher = better), not symmetric around zero. Forcing shorts on a directional signal is a design error.

Fix applied: Added `portfolio_mode` field to `AlphaConfig` with validation:

- `"long_only"` (default): `n_short=0`.
- `"long_short"`: requires `n_short > 0`.

Long-only is the correct default for a directional alpha on a 50-stock universe. Long-short makes sense only for market-neutral strategies with a genuinely symmetric signal.

Why Fixed N?

Fixed N has real advantages:

1. **Stable risk profile** — constant gross exposure, predictable diversification and capacity.
2. **No threshold to tune** — avoids the hyperparameter "what signal level is good enough?"
3. **Apples-to-apples** — isolates sizing effect from selection effect when comparing methods.
4. **Academic convention** — standard in factor evaluation (quintile L/S, decile spreads).

But it also has fundamental limitations:

- Stock ranked #11 gets zero weight regardless of signal strength (information discarded at selection).
- Forced positions when fewer than N stocks have genuinely positive signal.
- Signal magnitude is lost at the selection stage (Signal-Weighted partially recovers it at sizing).

The Portfolio Construction Spectrum

The rank-and-select approach is one end of a spectrum. The other end is **optimization-based** construction, where selection and sizing happen simultaneously:

Approach	Idea	Tradeoff
Equal-Weight	$1/N$ each position	Robust; ignores signal & risk
Inverse-Vol	$w_i \propto 1/\sigma_i$	Equal risk; needs σ estimates
Vol-Target	Scale to σ_{target}	Leverage varies; chases past vol
Signal-Weighted	$w_i \propto \lvert z_i \rvert/\sigma_i^2$	Uses conviction; ignores correlations
MVO	$w^* = \arg\max(w^\top \mu - \frac{\lambda}{2} w^\top \Sigma w)$	Optimal but fragile
Risk Parity	Equal risk contribution	Robust; ignores μ
Black-Litterman	Bayesian prior + views	Principled; complex

Mean-Variance Optimization (MVO)

The closed-form solution is:

$$w^* = \frac{1}{\lambda} \Sigma^{-1} \mu$$

where μ is the expected return vector (from the alpha signal), Σ is the covariance matrix, and λ is risk aversion.

Key insight: MVO solves selection and sizing *simultaneously*. Every stock gets a weight proportional to its signal strength, adjusted for correlation with the rest. No fixed N, no arbitrary threshold.

Practical problems with MVO:

1. **Covariance estimation** — 50 stocks $\Rightarrow$ 1,225 parameters in Σ. Mitigations: Ledoit–Wolf shrinkage, factor models.
2. **Extreme weights** — Σ^{-1} amplifies estimation error. Must add constraints ($|w_i| \leq w_{\max}$, leverage cap).
3. **Sensitivity to μ** — small changes in expected return $\Rightarrow$ completely different portfolio. Black-Litterman addresses this.
4. **Turnover explosion** — optimal portfolio changes daily. Add turnover penalty $\gamma \|w_t - w_{t-1}\|_1$ to the objective.

The classic quant aphorism: *"MVO is the world's best error maximizer"* — it takes the noisiest input (μ) and amplifies it through the most unstable operation (Σ^{-1}).

What This Means for Our Project

Our Phase 3 cost analysis proved the BBIBOLL + Vol Ratio alpha has a breakeven cost of $\sim$2 bps. MVO would not fix this — it would just allocate noise more "efficiently."

Priority order:

1. **Alpha improvement** (more factors, better combination) — fixes the real bottleneck.
2. **MVO implementation** — becomes worthwhile once the composite signal has breakeven $>$10 bps.
3. **Turnover-aware optimization** — internalize costs in the objective rather than analyzing post-hoc.

Interview Takeaway

"Position sizing is portfolio construction, which sits between alpha generation and execution. Rank-and-select is robust but throws away information. MVO is theoretically optimal but fragile — it amplifies estimation error in μ through Σ^{-1}. The right approach depends on signal quality: with IC $\approx$ 0.02, robust methods dominate; with IC $>$ 0.05, optimization starts to pay off. Either way, no portfolio construction method can rescue a weak alpha."

Appendix C

Glossary

Term	Definition
ADF	Augmented Dickey–Fuller test for stationarity
BBI	Bull and Bear Index (average of multiple SMAs)
CAGR	Compound Annual Growth Rate
CVaR	Conditional Value at Risk (Expected Shortfall)
FIGI	Financial Instrument Global Identifier
GARCH	Generalized Autoregressive Conditional Heteroskedasticity
GICS	Global Industry Classification Standard
IC	Information Coefficient
IR	Information Ratio
MDD	Maximum Drawdown
MVO	Mean-Variance Optimization (Markowitz)
OBV	On-Balance Volume
OHLCV	Open, High, Low, Close, Volume
OLS	Ordinary Least Squares
RV	Realized Volatility
VaR	Value at Risk
VWAP	Volume-Weighted Average Price
WFO	Walk-Forward Optimization

Table C.1: Common abbreviations used in this document

Appendix D

Learning Priority and Reading List

D.1 Recommended Order

1. **Data Pipeline** (Part I): Get clean data flowing first. Nothing else works without it.
2. **Backtest & Bias** (Part II): Build a strategy, measure it, understand why the measurement might be wrong.
3. **Factor IC/IR Analysis** (Part III, Ch. 9): The core language of quant interviews.
4. **Factor Construction** (Part III, Ch. 10): Normalization, neutralization — the preprocessing that makes or breaks a factor.
5. **Execution & Costs** (Part IV, Ch. 14): Makes your backtest realistic.
6. **Machine Learning** (Part V): Only after the linear factor framework is solid.
7. **Stochastic Calculus** (App. A.3): For derivatives and advanced risk; not urgent for equity alpha.

D.2 Suggested Reading

- Qian, Hua & Sorensen, *Quantitative Equity Portfolio Management* — the standard reference for factor investing.
- Grinold & Kahn, *Active Portfolio Management* — the "Fundamental Law of Active Management" and IC/IR framework.
- López de Prado, *Advances in Financial Machine Learning* — purged cross-validation, meta-labeling, feature importance.
- Chan, *Quantitative Trading* — practical guide for independent quant traders.
- Shreve, *Stochastic Calculus for Finance I & II* — the mathematical foundations if you want derivatives pricing.